

Technology Stack Details

Date:	08 October 2026	Team ID:	UNM1611JASON-SWTID-2026-4825
Project Name:	AI BlogNest API	Maximum Marks:	2 Marks

Technology Stack Architectural Selection:

A well-chosen technology stack ensures the application is performant, scalable, maintainable, and aligned with modern Generative AI software development practices.

Architecture Component / Layer Details — Part 1

S.No	Component / Layer	Technology Chosen	Justification / Purpose
1	Frontend / Client-Side (API Consumer)	• Thunder Client (VS Code Extension) • Postman Collection v2.1 • REST HTTP Web Client	• Enables rapid automated endpoint testing and regression suites. • Built-in environment variable management ({{{base_url}}}, {{{token}}}). • Lightweight, zero-overhead testing directly from the developer workspace.
2	Backend / Server-Side (Core API Framework)	• Node.js (v20+ LTS runtime) • Express.js (v4.18.2) • MVC Architecture Pattern	• Non-blocking, event-driven asynchronous I/O ideal for handling AI API latency. • Robust ecosystem, high throughput, and flexible middleware pipeline. • MVC pattern provides strict separation of concerns across controllers and models.

Technology Stack Details — Part 2

Architecture Component / Layer Details — Part 2

S.No	Component / Layer	Technology Chosen	Justification / Purpose
3	Database / Data Storage (Persistence Layer)	• MongoDB Atlas Cloud Cluster • Mongoose ODM (v7.5.0)	• Flexible schema design tailored for variable-length blog content and metadata. • Built-in schema validation, timestamps, and relational referencing via ObjectID. • High availability, automated indexing, and horizontal cloud scalability.
4	Cloud / Hosting / Deployment (Infrastructure)	• Node.js Cloud Runtime (Render) • Docker Container ready • Localhost (Development)	• Low-latency containerized execution with auto-restart capabilities. • Native environment secret injection via secure cloud dashboard. • Zero configuration continuous deployment connected directly to Git repository.
5	Version Control & CI/CD (Code Governance)	• Git & GitHub • Semantic Versioning (v1.0.0)	• Distributed version tracking, issue management, and collaborative development. • Clear commit traceability and clean branch management.

Technology Stack Details — Part 3 & Dependency Matrix

Architecture Component / Layer Details — Part 3

S.No	Component / Layer	Technology Chosen	Justification / Purpose
6	Third-Party APIs & Libraries (AI & Utilities)	• Google Generative AI SDK • jsonwebtoken (v9.0.2) • bcryptjs (v2.4.3) • express-validator (v7.0.1) • morgan, cors, dotenv	• @google/generative-ai: Direct interface to Gemini 1.5 Flash for state-of-the-art blog generation and summarization. • jsonwebtoken & bcryptjs: Industry-standard cryptographic auth and token validation. • express-validator: Strict sanitization and request body validation. • morgan & cors: HTTP request logging and cross-origin resource sharing.

Package Dependency & Version Manifest (package.json):

Package Name	Version	Type	Role in AI BlogNest API
@google/generative-ai	^0.24.1	Production	Official Google Gemini SDK for LLM generation & summarization
express	^4.18.2	Production	Core web framework for HTTP routing and middleware
mongoose	^7.5.0	Production	Object Data Modeling (ODM) for MongoDB database operations
jsonwebtoken	^9.0.2	Production	Generates and validates HMAC-SHA256 Bearer authorization tokens
bcryptjs	^2.4.3	Production	Salted cryptographic password hashing with 10 rounds
express-validator	^7.0.1	Production	Declarative request validation and error messaging middleware
cors & morgan	^2.8.5 / ^1.10.0	Production	Cross-Origin Resource Sharing & development HTTP logging
nodemon	^3.0.1	Dev Dependency	Automatic server reload upon file modifications during development